linux基础命令

点击跳转

> 1. cd 切换目录
> 2. touch 新建文件
> 3. ls 列出文件
> 4. mkdir 新建目录
> 5. echo 输出字符串
> 6. cat 查看文件
> 7. cp 复制文件
> 8. ln 新建链接
> 9. 权限
> 10. chmod 更改权限
> 11. chown 更改所属者

cd

1. `cd` 切换用户所在的目录

```
root@dell-PowerEdge-R740xd:\~# cd /opt/
root@dell-PowerEdge-R740xd:/opt#cd /opt/gitlab/
root@dell-PowerEdge-R740xd:/opt/gitlab#
```

touch

2. `touch` 新建一个空文件/将文件的访问和修改时间更新至当前
   `-a` 仅更改访问时间
   `-m` 仅更改修改时间

```
root@dell-PowerEdge-R740xd:\~# stat 123.txt
文件：123.txt
大小：0                 块：0          IO 块大小：4096   普通空文件
设备：8,2       Inode: 43550054    硬链接：1
权限：(0644/-rw-r--r--)  Uid: (    0/    root)   Gid: (    0/    root)
⚠️访问时间：2025-11-18 19:33:37⚠️.906071603 +0800
➡️修改时间：2025-11-18 10:58:10⬅️.895787463 +0800
变更时间：2025-11-18 19:33:37.906071603 +0800
创建时间：2025-11-18 10:57:41.817789076 +0800
root@dell-PowerEdge-R740xd:\~# touch -a 123.txt
root@dell-PowerEdge-R740xd:\~# stat 123.txt
文件：123.txt
 大小：0                 块：0          IO 块大小：4096   普通空文件
设备：8,2       Inode: 43550054    硬链接：1
权限：(0644/-rw-r--r--)  Uid: (    0/    root)   Gid: (    0/    root)
⚠️访问时间：2025-11-18 19:34:38⚠️.175068259 +0800  ⚠️#这里只有访问时间变了⚠️
➡️修改时间：2025-11-18 10:58:10⬅️.895787463 +0800
变更时间：2025-11-18 19:34:38.175068259 +0800
创建时间：2025-11-18 10:57:41.817789076 +0800
```

```
> root@dell-PowerEdge-R740xd:\~# touch 123.txt     *#新建空文件*
root@dell-PowerEdge-R740xd:\~# ls -hl 123.txt
-rw-r--r-- 1 root root 0 11月 18 ⚠️10:57⚠️123.txt
```

```
>root@dell-PowerEdge-R740xd:\~# touch 123.txt #更新时间(修改时间 访问时间)
root@dell-PowerEdge-R740xd:\~# ls -hl 123.txt
-rw-r--r-- 1 root root 0 11月 18 ⚠️10:58⚠️ 123.txt
```

ls

3. `ls` 列出目录下有什么

   #一般我用 `ll` ,这是alias `ll = 'ls -alF'` , `ls -alF` 的别名,直接 `ll` 省事

`-l` 长格式显示，包含权限、所有者、大小、时间等

```
root@dell-PowerEdge-R740xd:/home/dell# ls -l
总计 44
drwxr-xr-x 2 dell dell __4096__ 10月 29 12:19 公共
drwxr-xr-x 2 dell dell __4096__ 10月 29 12:19 模板
```

`-lh` 人类可读的格式显示文件大小

```
>root@dell-PowerEdge-R740xd:/home/dell#  ls -lh
总计 44K
drwxr-xr-x 2 dell dell **4.0K** 10月 29 12:19 公共
drwxr-xr-x 2 dell dell __4.0K__ 10月 29 12:19 模板
```

`-a` 显示所有文件，包括隐藏文件（以.开头的文件）

```
>root@dell-PowerEdge-R740xd:/home/dell# ls -a
.     桌面                __.gtkrc-2.0__                __.Xauthority__
..    __.bash_history__   __.ICEauthority__             __.xinitrc__
```

`-lt` 按修改时间排序（最新的在前）
`-lS` 按文件大小排序（最大的在前）
`-lr` 反向排序 (搭配 -S / -t)

mkdir

4. mkdir 新建文件夹

`mkdir dirname` 创建单个文件夹

```
root@dell-PowerEdge-R740xd:\~# mkdir test
root@dell-PowerEdge-R740xd:\~# ls -l
总计 4
1==这里的d代表的是目录==
drwxr-xr-x 2 root root 4096 11月 18 19:28 test
```

`mkdir -p` 递归创建多级目录 *#就是前面没有目录的自动新建一个*

```
root@dell-PowerEdge-R740xd:\~# `ls -a`
总计 0
root@dell-PowerEdge-R740xd:\~# `mkdir -p a/b`
root@dell-PowerEdge-R740xd:\~# `ls -R a`
总计 1
a/b:
```

`mkdir dir1 dir2 dir3` 一次性创建多个文件夹

```
root@dell-PowerEdge-R740xd:\~# `mkdir folder1 folder2 folder3`
root@dell-PowerEdge-R740xd:\~# `ls -l`
总计 12
drwxr-xr-x 2 root root 4096 11月 18 19:30 folder1
drwxr-xr-x 2 root root 4096 11月 18 19:30 folder2
drwxr-xr-x 2 root root 4096 11月 18 19:30 folder3
```

`mkdir -m` 创建目录时指定权限

```
root@dell-PowerEdge-R740xd:\~# `mkdir -m 755 public_dir`
root@dell-PowerEdge-R740xd:\~# `ls -l`
总计 4
==权限被设置为755==
drwxr-xr-x 2 root root 4096 11月 18 19:31 public_dir
```

echo

5. echo 输出字符串

```
root@dell-PowerEdge-R740xd:~# echo '你好'
你好
```

> 搭配重定向符号,将输出的文字写入文件 # *'>'是清空文件内容,再重定向 ,'>>'是把内容追加到文件末尾*

```
root@dell-PowerEdge-R740xd:~# touch 1.txt
root@dell-PowerEdge-R740xd:~# echo '你好' > 1.txt
root@dell-PowerEdge-R740xd:~# cat 1.txt
你好
root@dell-PowerEdge-R740xd:~# echo '你好' >> 1.txt
root@dell-PowerEdge-R740xd:~# cat 1.txt
你好
你好
```

cat

6. cat 查看文件内容

```
root@dell-PowerEdge-R740xd:~# cat 1.txt
你好
你好
```

也可以搭配'>'将文件追加到另外一文件

```
root@dell-PowerEdge-R740xd:~# echo '不好' > 2.txt
root@dell-PowerEdge-R740xd:~# cat 1.txt >> 2.txt
root@dell-PowerEdge-R740xd:~# cat 2.txt
不好
你好
你好
```

cp

7. cp 复制文件和目录

`cp source dest` 复制源文件到目标位置

```
cp file1.txt file2.txt
```

`cp -i` 交互式复制，覆盖前提示

```
cp -i file1.txt file2.txt
```

`cp -r` 递归复制目录及其内容

```
cp -r dir1 dir2
```

`cp -p` 保留原文件的属性(权限、时间戳等)

```
cp -p file1.txt file3.txt
```

`cp -v` 显示详细的复制过程

```
cp -v file1.txt file4.txt
```

`cp -u` 更新模式，仅当源文件比目标文件新时才复制

```
cp -u file1.txt file2.txt
```

`cp -l` 创建硬链接而非复制文件内容 软 / 硬连接说明

```
cp -l file1.txt file_hardlink.txt
```

`cp -s` 创建符号链接（软链接） 软 / 硬连接说明

```
cp -s file1.txt file_softlink.txt
```

`cp -f` 强制复制，覆盖已存在的目标文件而不提示

```
cp -f file1.txt file2.txt
```

`cp -t` 指定目标目录，源文件放在最后

```
cp -t /target/dir/ file1.txt
```

ln

8. ln 创建文件链接

`ln` 创建硬链接

```
ln file1.txt file1_hardlink.txt
```

`ln -s` 创建符号链接（软链接）

```
ln -s file1.txt file1_softlink.txt
```

`ln -f` 强制创建链接，覆盖已存在的目标文件

```
ln -f file1.txt existing_link.txt
```

`ln -v` 显示详细的链接创建过程

```
ln -v file1.txt file1_link.txt
```

`ln -t` 指定链接文件的目标目录

```
ln -t /home/user/links/ file1.txt file2.txt
```

`ln -P` 直接创建硬链接到符号链接（而不是解引用）

```
ln -P symlink_source hardlink_to_symlink
```

`ln -L` 解引用所有遇到的符号链接

```
ln -L symlink_source new_hardlink
```

软硬连接说明

🤔🐟假设我们有如下文件结构：🤔🐟

```
实际文件.txt  -> 内容为 "Hello World"
符号链接_A    -> 指向 ‘实际文件.txt‘  (软链接)
```

`ln -P 符号链接_A 硬链接_B`

操作对象：符号链接_A 这个文件本身。

结果：硬链接_B 是 符号链接_A 的硬链接。现在有两个符号链接文件（符号链接_A 和 硬链接_B），它们都存储着相同的路径信息 "实际文件.txt"。如果你删除 实际文件.txt，这两个符号链接都会变成“悬空”的坏链接。

`ln -L 符号链接_A 硬链接_C`

操作对象：符号链接_A 最终指向的 实际文件.txt。

结果：硬链接_C 是 实际文件.txt 的硬链接。现在 实际文件.txt 和 硬链接_C 指向同一个⚠️数据块⚠️。符号链接_A 只是一个中间人。

`ln -s 符号链接_A 符号链接_D`

操作对象：创建一个新的符号链接。

结果：符号链接_D -> 符号链接_A -> 实际文件.txt

	软连接	**硬链接**
特性	硬链接	软连接
本质	同一个文件的多个名字	指向文件的快捷方式
inode	与原文件相同	新的inode
磁盘空间	不额外占用	占用少量空间存储路径
原文件删除	数据仍存在	链接失效
跨文件系统	❌ 不支持	✅ 支持
指向目录	❌ 不支持	✅ 支持
文件类型	普通文件	链接文件 (l)

权限

9. 文件 / 目录权限

文件的权限针对三类对象进行定义

- 文件权限
  owner 属主，缩写u
  group 属组，缩写g
  other 其他，缩写o
- 每个文件针对每类访问者定义了三种主要权限
  r：Read 读 (4) <--这里的数字后面 学到
  w：Write 写 (2) <--这里的数字后面 学到
  x：eXecute 执行 (1) <--这里的数字后面 学到

另 X：针对目录加执行权限,文件不加执行权限(因文件具备执行权限有安全隐患)

现在看一个权限被赋予'777'的文件

```
drwxrwxrwx  5 root root 4.0K 11月 14 15:08 venv


        所有者(Owner)   用户组(Group)  其他人(Others)
         |    |    |    |    |    |    |    |    |
    d    r    w    x    r    w    x    r    w    x
    ↑    ↑    ↑    ↑    ↑    ↑    ↑    ↑    ↑    ↑
  目录  读取 更改  执行 读取 编写  执行 读取 更改 执行
```

有的文件是链接文件所以第一个字母会是'l'

```
drwxr-xr-x  ⚠5⚠ root root
```

然后可以看到后面的 `5`
这个 `5` 表示的是 **链接数**.

- 对于**普通文件**来说，链接数表示有多少个文件名指向这个文件（**硬链接**的数量）。
- 对于**目录**来说，链接数的含义稍有不同：
  - 每个目录至少有两个链接：
    a. 自身的条目（比如 /tmp 的 .）。
    b. 其父目录中指向它的条目（例如 /root/venv的root目录就是父目录）。
  - 此外，目录中每一个直接**子目录**都会为父目录**增加**一个链接。因为每个子目录里都有一个 .. 条目指向父目录(就是子目录的'..'指向父目录,就生成了一个链接)。

我们可以 `ll ../venv/`

```
总计 24
drwxr-xr-x  5 root root 4096 11月 17 16:36 ./              <-- 1
drwx------ 23 root root 4096 11月 19 21:24 ../             <-- 2
drwxr-xr-x  2 root root 4096 11月 17 20:14 bin/            <-- 3
drwxr-xr-x  4 root root 4096 11月 17 16:40 include/        <-- 4
drwxr-xr-x  3 root root 4096 11月 17 16:36 lib/            <-- 5
lrwxrwxrwx  1 root root    3 11月 17 16:36 lib64 -> lib/   <-- 这个软连接指向lib的不算
-rw-r--r--  1 root root  149 11月 17 16:36 pyvenv.cfg      <-- 文件不算
```

所以是 `5` .

再后面看到的 `root root`
这里分别是 `Owner(所有者)` `Group(所属组)`
如果权限是 `-rw-r--r--`
那么 `所有者(root)` 可以读取(r)和更改(w), `所属组(root)` 只能读取(r)

chmod

10. chmod 更改文件权限

`chmod` 权限 文件

1. 符号模式 （ugoa+/-权限）
   [u g o a][+ - =][r w x X]

- 其中:
  `u` 表示该文件的拥有者
  `g` 表示与该文件的拥有者属于同一个群体(group)者
  `o` 表示其他以外的人
  `a` 表示这三者皆是。

  `+` 表示增加权限、 `-` 表示取消权限、 `=` 表示唯一设定权限。

  `r` 表示可读取， `w` 表示可写入， `x` 表示可执行， `X` 表示只有当该文件是个子目录或者该文件已经被设定过为可执行。
  关于'X'的解释
  i. 对于 `目录` ：
  总是会添加执行权限（x）,因为目录需要执行权限才能进入

ii. 对于 文件 ：
只有当该文件已经至少有一个执行权限位被设置时，才会添加执行权限
或者当前用户类别（用户、组、其他）原本就有执行权限

总结:对于 文件 ,那就是如果权限是-rwxr--r--可以加g+X,而权限是-rw-r--r--的g+X则加不了执行. 对于 目录 ,总是会添加执行权限（x）,因为目录需要执行权限才能进入

```
# 安全地给脚本目录添加执行权限
chmod -R a+rX scripts/  # 只给真正的可执行文件添加执行权限

# 这比下面这个更安全，不会给文本文件等添加执行权限
chmod -R a+rx scripts/  # 可能给所有文件都加了执行权限
```

2. 八进制语法 通过给数字来赋予权限(我爱用)

数字	权限
4	读(r)
2	写(w)
1	执行(x)

文件或目录的权限位是由 9 个权限位来控制， 每三位为一组 ，它们分别是
文件所有者 （User）的读、写、执行
用户组 （Group）的读、写、执行
其它用户 （Other）的读、写、执行

#	权限	rwx	二进制
7	读 + 写 + 执行	rwx	111
6	读 + 写	rw-	110
5	读 + 执行	r-x	101
4	只读	r--	100
3	写 + 执行	-wx	011
2	只写	-w-	010
1	只执行	--x	001
0	无	---	000

```
root@dell-PowerEdge-R740xd:~# ll 1.txt
⚠-rw-r--r--⚠ 1 root root 14 11月 19 10:15 1.txt
root@dell-PowerEdge-R740xd:~# chmod 744 1.txt
root@dell-PowerEdge-R740xd:~# ll 1.txt
⚠-rwxr--r--⚠ 1 root root 14 11月 19 10:15 1.txt
```

chown

11. chown 更改文件的所有者
    #本来还有chgrp的但是没必要,chown也能改所属组

```
chown  用户名(:组名)  文件名
```

更改文件所属 人

```
(venv) root@dell-PowerEdge-R740xd:~# ll 1.txt
-rwxr--r-- 1 ⚠root⚠ root 14 11月 19 10:15 1.txt
(venv) root@dell-PowerEdge-R740xd:~# chown dell 1.txt
(venv) root@dell-PowerEdge-R740xd:~# ll 1.txt
-rwxr--r-- 1 ⚠dell⚠ root 14 11月 19 10:15 1.txt
```

更改文件所属 组

```
(venv) root@dell-PowerEdge-R740xd:~# ll 1.txt
-rwxr--r-- 1 dell ⚠root⚠ 14 11月 19 10:15 1.txt
(venv) root@dell-PowerEdge-R740xd:~# chown :dell 1.txt  #前面加个冒号
(venv) root@dell-PowerEdge-R740xd:~# ll 1.txt
-rwxr--r-- 1 dell ⚠dell⚠ 14 11月 19 10:15 1.txt
```

全都改

```
(venv) root@dell-PowerEdge-R740xd:~# ll 1.txt
-rwxr--r-- 1 ⚠dell dell⚠ 14 11月 19 10:15 1.txt*
(venv) root@dell-PowerEdge-R740xd:~# chown root:root 1.txt
(venv) root@dell-PowerEdge-R740xd:~# ll 1.txt
-rwxr--r-- 1 ⚠root root⚠ 14 11月 19 10:15 1.txt*
```